

Zisk Prover Specification

Derived from the Python Executable Specification

Abstract

This document is the complete specification for the PIL2 proving system as used by the Zisk zkVM. It specifies *exactly* what the prover and verifier compute, in sequence, using mathematical notation. It contains no proofs and no security analysis—only the concrete protocol.

The specification comprises three parts:

1. **STARK Protocol** — the parametric FRI-STARK proving system that works with any AIR over the Goldilocks field.
2. **Zisk Machine** — the concrete chip architecture: 21 AIRs, bus interconnections, memory layout, coprocessors, precompile circuits, lookup tables, and global constraints.
3. **Recursion Pipeline** — the VADCOP recursive aggregation from per-AIR STARKs to a single Ethereum-verifiable proof.

This specification is accompanied by a Python executable specification that implements the complete STARK prover and verifier, producing byte-identical proofs to the production C++ implementation.

Contents

I	STARK Protocol	4
1	Notation and Algebraic Setup	5
1.1	Fields	5
1.2	Domains	5
1.3	Polynomials	5
1.4	Key Quantities	5
1.5	Notation Conventions	6
2	Building Blocks	6
2.1	Polynomial Commitment via Merkle Trees	6
2.2	Fiat-Shamir Transcript	6
3	STARK Protocol: Commitment Phase	7
3.1	Stage 1: Witness Commitment	8
3.2	Transcript Seeding	8
3.3	Stage 2: Intermediate Polynomials	9
3.4	Stage Q: Quotient Polynomial	11
3.5	Polynomial Evaluations	11
3.6	FRI Polynomial Construction	12
3.7	FRI Commitment Rounds	12
3.8	Proof of Work	13
4	STARK Protocol: Query Phase	13
4.1	Transcript Reconstruction	14
4.2	Constraint Check	14
4.3	Proof-of-Work Check	15
4.4	Final Polynomial Degree Check	15
4.5	Query Derivation	15
4.6	Merkle Tree Verification	15
4.7	FRI Polynomial Consistency	15
4.8	FRI Folding Verification	16
5	Multi-AIR Challenge Binding	16
5.1	Per-AIR Contributions	16
5.2	Challenge Aggregation	17
5.3	Global Constraints	17
II	Zisk Machine	18
6	Architecture Overview	19
6.1	Airgroup Structure	19
6.2	AIR Inventory	19
6.3	STARK Parameters per AIR	20
7	Bus Architecture	21
7.1	Bus Types	21
7.2	Bus Inventory	22

	7.3	Bus Interconnection Summary	22
8		CPU: Main AIR	22
	8.1	Instruction Execution	22
	8.2	Bus Interactions	23
9		Memory Subsystem	23
	9.1	Mem AIR	23
	9.2	RomData AIR	23
	9.3	InputData AIR	23
	9.4	MemAlign Subsystem	23
10		Computation Coprocessors	24
	10.1	Binary	24
	10.2	BinaryAdd	24
	10.3	BinaryExtension	24
	10.4	Arith	24
11		Precompiles	24
	11.1	Add256	24
	11.2	ArithEq	24
	11.3	ArithEq384	24
	11.4	Keccakf	25
	11.5	Sha256f	25
12		Lookup Tables	25
	12.1	SpecifiedRanges	25
	12.2	VirtualTable0	25
	12.3	VirtualTable1	25
13		Global Constraints	25
	13.1	Bus Balance Equations	26
	13.2	Continuation Anchoring	26
	13.3	Airgroup Value Aggregation	26
III Recursion Pipeline			27
14		Pipeline Overview	28
	14.1	Proof Type Table	28
	14.2	Field Transitions	29
15		Basic STARK Proof (Stage 1)	29
16		Compressor (Stage 2, Optional)	30
17		Recursive1: Per-AIR Normalization (Stage 3)	30
18		Recursive2: Tree Aggregation (Stage 4)	30
	18.1	Aggregation Structure	31
	18.2	Airgroup Value Aggregation	31
	18.3	stage1Hash Chaining	31
	18.4	Tree Depth	31
19		VadcopFinal (Stage 5)	31
	19.1	STARK Verification	31
	19.2	Global Challenge Recomputation	32
	19.3	Global Constraint Verification	32
	19.4	Output	32
20		RecursiveF: Field Transition (Stage 6, Not Deployed)	32
21		fflonk: Ethereum-Verifiable Proof (Stage 7, Not Deployed)	33
22		Distributed Proving	33

IV	Appendices	34
A	Constraint Polynomial Structure	35
B	FRI Polynomial Batching Formula	35

Part I

STARK Protocol

1 Notation and Algebraic Setup

1.1 Fields

The base field is the Goldilocks prime field

$$\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}, \quad p = 2^{64} - 2^{32} + 1.$$

The cubic extension field is

$$\mathbb{F}_{p^3} = \mathbb{F}_p[\alpha]/(\alpha^3 - \alpha - 1),$$

where α is a root of the irreducible polynomial $X^3 - X - 1$ over $\mathbb{F}_p$. An element of $\mathbb{F}_{p^3}$ is written as $a_0 + a_1\alpha + a_2\alpha^2$ with $a_i \in \mathbb{F}_p$.

1.2 Domains

Let $N = 2^n$ be the *trace size* (number of rows in the execution trace). The *trace domain* is

$$H = \{\omega^i : i = 0, \dots, N-1\},$$

where $\omega \in \mathbb{F}_p$ is a primitive N -th root of unity.

The *extended evaluation domain* is a coset

$$H^* = \{g \cdot \omega_{\text{ext}}^i : i = 0, \dots, N_{\text{ext}} - 1\},$$

where $g = 7 \in \mathbb{F}_p$ is the coset shift, ω_{ext} is a primitive N_{ext} -th root of unity, and $N_{\text{ext}} = 2^{n_{\text{ext}}}$. The *blowup factor* is $\beta = N_{\text{ext}}/N$.

1.3 Polynomials

- *Witness polynomials* $f_1, \dots, f_m$: stage 1 committed columns (execution trace), each of degree $< N$.
- *Intermediate polynomials* $h_1, \dots, h_k$: stage 2 committed columns (lookup / permutation support), degree $< N$.
- *Constant polynomials* $c_1, \dots, c_\ell$: fixed by the AIR definition, committed during setup.
- *Quotient polynomial* Q : the result of dividing the constraint polynomial by the vanishing polynomial.
- *FRI polynomial* F : a linear combination of all committed polynomials used as input to FRI.

1.4 Key Quantities

Symbol	Meaning
$Z_H(X) = X^N - 1$	Vanishing polynomial on H
$\mathcal{O} = \{o_0, o_1, \dots\}$	Opening point offsets (typically $\subseteq \{-1, 0, 1\}$)
J	Number of constraint polynomials in the AIR
d	Number of quotient polynomial pieces (Q split degree)
K	Number of FRI folding rounds
Q_{queries}	Number of FRI query repetitions
b_{pow}	Proof-of-work difficulty (number of leading zero bits)
a	Merkle tree arity (2, 3, or 4)

1.5 Notation Conventions

Notation	Meaning
$[n]$	The set $\{0, 1, \dots, n-1\}$
$\text{MT}(\cdot)$	Merkle tree root
$\mathcal{T}$	Fiat-Shamir transcript
$\mathcal{T}.\text{absorb}(\cdot)$	Absorb elements into transcript
$\mathcal{T}.\text{squeeze}()$	Squeeze one $\mathbb{F}_{p^3}$ challenge
$\mathcal{T}.\text{squeeze_indices}(q, b)$	Squeeze q pseudorandom b -bit indices
$\xi, \beta_k, v_1, v_2, v_c$	Challenges in $\mathbb{F}_{p^3}$
α, γ	Lookup/permutation challenges in $\mathbb{F}_{p^3}$

2 Building Blocks

2.1 Polynomial Commitment via Merkle Trees

A polynomial f of degree $< N$ is committed as follows.

1. **Extend to evaluation domain.** Compute coefficients $\hat{f} = \text{INTT}_N(f)$, zero-pad to N_{ext} coefficients, and evaluate on the coset domain: $\tilde{f} = \text{NTT}_{N_{\text{ext}}}(\hat{f})$. This yields N_{ext} evaluations $\tilde{f}(x)$ for $x \in H^*$.
2. **Build Merkle tree.** When committing multiple polynomials $f_1, \dots, f_w$ jointly:
 - (a) Form row i as the concatenation of all polynomial values at domain point i : $\ell_i = (\tilde{f}_1(x_i), \tilde{f}_2(x_i), \dots, \tilde{f}_w(x_i))$.
 - (b) Hash each row: $h_i = \text{LinearHash}(\ell_i)$ using Poseidon2 linear hashing.
 - (c) Build an arity- a Merkle tree over leaves $h_0, \dots, h_{N_{\text{ext}}-1}$.
3. **Output.** The commitment is the Merkle root $r = \text{MT}(\tilde{f}_1, \dots, \tilde{f}_w)$.

Opening proof at index j . The prover reveals the leaf values ℓ_j together with the authentication path (sibling hashes along the path from leaf j to the root).

Verification. The verifier recomputes $h_j = \text{LinearHash}(\ell_j)$, walks the authentication path using the provided siblings, and checks that the computed root matches the committed root r .

2.2 Fiat-Shamir Transcript

The transcript $\mathcal{T}$ is a Poseidon2 sponge with width $w = 4a$, where $a \in \{2, 3, 4\}$ is the sponge arity. The state is partitioned into a *rate* portion of $4(a-1)$ elements and a *capacity* portion of 4 elements.

Operations.

- $\mathcal{T}.\text{absorb}(x_1, \dots, x_k)$: Feed field elements into the rate portion one at a time, applying the Poseidon2 permutation whenever the rate portion is full.
- $\mathcal{T}.\text{squeeze}() \rightarrow (c_0, c_1, c_2) \in \mathbb{F}_{p^3}$: Squeeze three base-field elements from the sponge output (applying a permutation if needed), interpreted as a single extension-field challenge $c_0 + c_1\alpha + c_2\alpha^2$.

- $\mathcal{T}.\text{squeeze_indices}(q, b) \rightarrow (i_1, \dots, i_q)$: Squeeze enough field elements to extract q pseudo-random b -bit indices via bit packing. Specifically, squeeze $\lceil q \cdot b/63 \rceil$ field elements and extract b bits per index from the binary representations, using 63 bits per field element.

3 STARK Protocol: Commitment Phase

The commitment phase is the first half of the STARK protocol. The prover commits to polynomials in stages, derives challenges via the Fiat-Shamir transcript, and constructs the FRI polynomial. In the non-interactive version, all verifier messages are derived deterministically from the transcript $\mathcal{T}$.

Table 1: PIL2-STARK: Commitment Phase

Prover	Verifier
<i>Stage 1: Witness (Section 3.1)</i>	
Hold witness polynomials $f_1, \dots, f_m$	
Extend each f_j to H^* , commit:	$\longrightarrow r_1$
$r_1 = \text{MT}(f_1, \dots, f_m)$	
	Seed transcript $\mathcal{T}$ from $(\text{vk}, \text{pub}, r_1)$
<i>Stage 2: Intermediate polynomials (Section 3.3)</i>	
$(\alpha, \gamma, \dots)$	$\longleftarrow \alpha, \gamma, \dots \leftarrow \mathcal{T}.\text{squeeze}()$
Compute intermediate columns $h_1, \dots, h_k$ using challenges α, γ	
Extend each h_j to H^* , commit:	$\longrightarrow r_2$
$r_2 = \text{MT}(h_1, \dots, h_k)$	
	$\mathcal{T}.\text{absorb}(r_2)$
<i>Stage Q: Quotient polynomial (Section 3.4)</i>	
v_c	$\longleftarrow v_c \leftarrow \mathcal{T}.\text{squeeze}()$
Evaluate combined constraint on H^* :	
$C(x) = \sum_{j=0}^{J-1} v_c^{J-1-j} C_j(x)$	
Compute $\bar{Q}(x) = C(x)/Z_H(x)$	
Split:	
$Q = Q_0 + X^N Q_1 + \dots + X^{(d-1)N} Q_{d-1}$	
Commit: $r_Q = \text{MT}(Q_0, \dots, Q_{d-1})$	$\longrightarrow r_Q$
	$\mathcal{T}.\text{absorb}(r_Q)$
<i>Evaluation stage (Section 3.5)</i>	
ξ	$\longleftarrow \xi \leftarrow \mathcal{T}.\text{squeeze}()$
For each polynomial p and offset $o \in \mathcal{O}$:	
$e_{p,o} = p(\xi \cdot \omega^o) \in \mathbb{F}_{p^3}$	
$\{e_{p,o}\}$	$\longrightarrow \{e_{p,o}\}$
	$\mathcal{T}.\text{absorb}(\text{LinearHash}(\{e_{p,o}\}))$
<i>FRI polynomial (Section 3.6)</i>	
(v_1, v_2)	$\longleftarrow v_1, v_2 \leftarrow \mathcal{T}.\text{squeeze}()$
Compute FRI polynomial F on H^* :	
$F(x) =$	
$\sum_g v_1^{ \mathcal{G} -1-g} \cdot \frac{\sum_j v_2^{n_g-j} (p_j(x) - e_j)}{x - \xi \omega^{o_g}}$	

FRI commitment rounds (Section 3.7)

Set $F_0 = F$

Commit: $r_0^{\text{FRI}} = \text{MT}(F_0)$	$\longrightarrow$	r_0^{FRI} $\mathcal{T}.\text{absorb}(r_0^{\text{FRI}})$
β_0	$\longleftarrow$	$\beta_0 \leftarrow \mathcal{T}.\text{squeeze}()$
Fold: $F_1 = \text{Fold}(F_0, \beta_0)$		
$\vdots$		$\vdots$
Commit: $r_{K-1}^{\text{FRI}} = \text{MT}(F_{K-1})$	$\longrightarrow$	r_{K-1}^{FRI} $\mathcal{T}.\text{absorb}(r_{K-1}^{\text{FRI}})$
β_{K-1}	$\longleftarrow$	$\beta_{K-1} \leftarrow \mathcal{T}.\text{squeeze}()$
Fold: $F_K = \text{Fold}(F_{K-1}, \beta_{K-1})$		
F_K	$\longrightarrow$	F_K $\mathcal{T}.\text{absorb}(\text{LinearHash}(F_K))$ Check: $\deg(F_K) < D$

Proof of work (Section 3.8)

χ_{grind}	$\longleftarrow$	$\chi_{\text{grind}} \leftarrow \mathcal{T}.\text{squeeze}()$
Find η such that $\text{Poseidon2}(\chi_{\text{grind}} \parallel \eta)$ has b_{pow} leading zero bits		
η	$\longrightarrow$	η Check: $\text{Poseidon2}(\chi_{\text{grind}} \parallel \eta)$ has b_{pow} leading zeros

Note on the non-interactive setting. The protocol is presented in the interactive model for clarity. In the actual (non-interactive) protocol, the verifier has no role during the commitment phase—the right column shows transcript operations that the prover executes locally and that the verifier will *replay* during the query phase (Section 4.1) to rederive all challenges from the proof data. All substantive verifier checks (constraint verification, Merkle proofs, FRI consistency) occur in the query phase (Table 2).

The following subsections detail each stage shown in the diagram above.

3.1 Stage 1: Witness Commitment

The prover holds witness polynomials $f_1, \dots, f_m : H \rightarrow \mathbb{F}_p$ representing the execution trace columns.

- (i) Extend each f_j to the evaluation domain H^* .
- (ii) Build a joint Merkle tree over all stage-1 columns.
- (iii) Output the commitment $r_1 = \text{MT}(f_1, \dots, f_m)$.

3.2 Transcript Seeding

Initialize the Fiat-Shamir transcript. The seeding depends on the protocol mode:

- **Multi-AIR (VADCOP) mode.** A global challenge $\chi \in \mathbb{F}_{p^3}$ is derived from the verification key, public inputs, and r_1 via a lattice expansion procedure (see Section 5). Seed the transcript:

$$\mathcal{T}.\text{absorb}(\chi).$$

- **Standalone mode.** Seed the transcript directly:

$$\mathcal{T}.\text{absorb}(\text{vk}, H(\text{pub}), r_1),$$

where vk is the verification key (4 base-field elements), $\text{H}(\text{pub})$ is a Poseidon2 hash of the public inputs, and r_1 is the stage-1 Merkle root.

3.3 Stage 2: Intermediate Polynomials

- (i) Derive stage-2 challenges from the transcript:

$$\alpha, \gamma \leftarrow \mathcal{T}.\text{squeeze()}.$$

(One `squeeze()` call per challenge specified by the AIR.)

- (ii) Compute intermediate columns $h_1, \dots, h_k$ using the challenges. The types of intermediate columns are enumerated below.
- (iii) Extend and commit: $r_2 = \text{MT}(h_1, \dots, h_k)$.
- (iv) Absorb: $\mathcal{T}.\text{absorb}(r_2)$.

Challenges and the compress function. Two extension-field challenges drive all stage-2 computations:

- $\alpha \in \mathbb{F}_{p^3}$: random linear combination challenge (Schwartz–Zippel style);
- $\gamma \in \mathbb{F}_{p^3}$: offset challenge preventing zero denominators.

Given a bus identifier $b \in \mathbb{F}_p$ and column values $c_1, \dots, c_w$, the *compressed expression* is

$$\text{compress}(b, c_1, \dots, c_w) = (\dots((c_w \cdot \alpha + c_{w-1}) \cdot \alpha + \dots) \cdot \alpha + b) + \gamma. \quad (1)$$

This maps a tuple $(b, c_1, \dots, c_w)$ to a single element of $\mathbb{F}_{p^3}$.

Logup terms. Each AIR defines a set of *logup terms*, each consisting of a bus identifier, column tuple, and a stored numerator. A term can be a *provider* (numerator = +multiplicity, the AIR that supplies the lookup entry) or a *consumer* (numerator = −selector, the AIR that uses the lookup entry). The logup argument ensures that $\sum_i s_i/D_i = 0$ across all AIRs sharing the same bus, where $D_i = \text{compress}(b_i, \mathbf{c}_i)$.

Intermediate column types. All stage-2 columns fall into the following types, generated by the PIL standard library (`std_sum` for logup/sum arguments, `std_prod` for permutation/product arguments) or by AIR-specific PIL code.

Type	Description
<code>im_cluster</code>	Batched logup: combines ≥ 2 terms into one column
<code>im_single</code>	Single logup term that cannot be batched
<code>gsum</code>	Running sum accumulator (logup argument)
<code>gprod</code>	Running product accumulator (permutation argument)
<code>im_low</code>	Degree reduction for product argument terms
<code>ImPol</code>	AIR-specific constraint degree reduction

All columns below are over $\mathbb{F}_{p^3}$ unless noted otherwise.

Clustered logup intermediate (im_cluster)

When the compiler groups t logup terms into a single column, the clustered intermediate is defined by the constraint

$$\text{im_cluster} \cdot \prod_{j=1}^t D_j = \sum_{j=1}^t s_j \cdot \prod_{\substack{i=1 \\ i \neq j}}^t D_i,$$

where $D_j = \text{compress}(b_j, \mathbf{c}_j)$ and s_j is the stored numerator for term j . In the common $t = 2$ case this reduces to

$$\text{im_cluster} = \frac{s_1 \cdot D_2 + s_2 \cdot D_1}{D_1 \cdot D_2}.$$

Semantically, im_cluster equals $\sum_j s_j / D_j$ (the sum of individual logup contributions), combined into a single column to reduce the number of committed polynomials.

Single logup intermediate (im_single)

When a logup term cannot be grouped with others (e.g. because the combined degree would exceed the constraint degree bound), it gets its own column:

$$\text{im_single} = \frac{s}{D}, \quad \text{where } D = \text{compress}(b, \mathbf{c}).$$

Grand sum (gsum)

The running cumulative sum of all logup contributions at each row:

$$\text{gsum}[i] = \sum_{j=0}^i \left(\sum_{\ell} \text{im_cluster}_{\ell}[j] + \sum_{\ell} \text{im_single}_{\ell}[j] + \frac{s_0[j]}{D_0[j]} \right),$$

where the last term is the “direct” logup contribution (one term that enters the sum without an intermediate column). The boundary constraint at the final row asserts that $\text{gsum}[N-1]$ equals a declared *airgroup value*, which is later checked for cross-AIR consistency (see Section 5.3).

Grand product (gprod)

For the permutation argument, a running cumulative product:

$$\text{gprod}[0] = \frac{n_0}{d_0}, \quad \text{gprod}[i] = \text{gprod}[i-1] \cdot \frac{n_i}{d_i} \quad \text{for } i > 0,$$

where the numerator and denominator for each row are products of terms of the form $\text{sel} \cdot (\text{compress}(b, \mathbf{c}) - \gamma + \gamma - 1) + 1$. As with gsum , a boundary constraint at the final row checks the accumulated product against an *airgroup value*.

Low-degree reduction (im_low)

When a product argument has too many terms whose combined degree exceeds the constraint degree bound, the compiler introduces im_low columns that hold partial product ratios:

$$\text{im_low} = \frac{\prod_j n_j}{\prod_j d_j},$$

where $\{n_j\}$ and $\{d_j\}$ are subsets of the numerator and denominator terms. The gprod recurrence then references im_low instead of the raw product terms.

Application-level intermediates (ImPol)

Individual AIRs may declare their own stage-2 columns to reduce constraint degree. For example, if an AIR constraint has the form $A \cdot B \cdot C \cdot D = E$ and the product's degree exceeds the bound, the AIR author introduces a column $\text{ImPol} = A \cdot B$ and checks $\text{ImPol} \cdot C \cdot D = E$. These may be over the base field $\mathbb{F}_p$ or the extension $\mathbb{F}_{p^3}$, depending on whether the constraint involves challenges.

3.4 Stage Q: Quotient Polynomial

- (i) Derive the constraint-combination challenge:

$$v_c \leftarrow \mathcal{T}.\text{squeeze}().$$

- (ii) Evaluate the combined constraint polynomial on the extended domain. Let J be the number of individual constraint polynomials defined by the AIR (see Appendix A). For each $x \in H^*$:

$$C(x) = v_c^{J-1} \cdot C_0(x) + v_c^{J-2} \cdot C_1(x) + \cdots + C_{J-1}(x). \quad (2)$$

This is computed via Horner's method.

- (iii) Compute the quotient polynomial on the extended domain:

$$Q(x) = \frac{C(x)}{Z_H(x)} = \frac{C(x)}{x^N - 1} \quad \text{for } x \in H^*.$$

- (iv) Split Q into d pieces of degree $< N$:

$$Q(X) = Q_0(X) + X^N \cdot Q_1(X) + \cdots + X^{(d-1)N} \cdot Q_{d-1}(X). \quad (3)$$

Concretely:

- (a) Convert Q from evaluation form to coefficient form via $\text{INTT}_{N_{\text{ext}}}$.
- (b) Extract coefficients: Q_j has coefficients $\hat{q}_{jN}, \hat{q}_{jN+1}, \dots, \hat{q}_{(j+1)N-1}$.
- (c) Apply coset correction: multiply each Q_j 's coefficients by $S_j = g^{-jN}$.
- (d) Extend each Q_j to the evaluation domain via $\text{NTT}_{N_{\text{ext}}}$.
- (v) Commit all pieces jointly: $r_Q = \text{MT}(Q_0, \dots, Q_{d-1})$.
- (vi) Absorb: $\mathcal{T}.\text{absorb}(r_Q)$.

3.5 Polynomial Evaluations

- (i) Derive the evaluation challenge:

$$\xi \leftarrow \mathcal{T}.\text{squeeze}().$$

This is an element of $\mathbb{F}_{p^3}$.

- (ii) For each committed polynomial p and each opening offset $o \in \mathcal{O}$, compute the evaluation

$$e_{p,o} = p(\xi \cdot \omega^o) \in \mathbb{F}_{p^3}. \quad (4)$$

This includes witness polynomials (f_j), intermediate polynomials (h_j), quotient pieces (Q_j), and constant polynomials (c_j).

- (iii) Absorb a hash of all evaluations:

$$\mathcal{T}.\text{absorb}(\text{LinearHash}(e_{p,o} \text{ for all } p, o)).$$

3.6 FRI Polynomial Construction

- (i) Derive the batching challenges:

$$v_1, v_2 \leftarrow \mathcal{T}.\text{squeeze}().$$

- (ii) Group all evaluation-map entries by their opening offset index g . Let $\mathcal{G} = \{g_0, g_1, \dots\}$ be the ordered set of opening-offset indices.
- (iii) For each group g with entries $\{(p_0, e_0), (p_1, e_1), \dots, (p_{n_g}, e_{n_g})\}$, compute the group polynomial via Horner accumulation with v_2 :

$$G_g(x) = \frac{1}{x - \xi \cdot \omega^{o_g}} \cdot \left(v_2^{n_g} (p_0(x) - e_0) + v_2^{n_g-1} (p_1(x) - e_1) + \dots + (p_{n_g}(x) - e_{n_g}) \right), \quad (5)$$

where o_g is the opening offset for group g .

- (iv) Combine all groups via Horner accumulation with v_1 :

$$F(x) = v_1^{|\mathcal{G}|-1} \cdot G_{g_0}(x) + v_1^{|\mathcal{G}|-2} \cdot G_{g_1}(x) + \dots + G_{g_{|\mathcal{G}|-1}}(x). \quad (6)$$

- (v) Evaluate F on the extended domain H^* to obtain the FRI input polynomial.

See Appendix B for the complete batching formula.

3.7 FRI Commitment Rounds

The FRI protocol reduces the degree of F through iterated folding. Let $F_0 = F$ and $b_0 = n_{\text{ext}}$ be the initial domain size in bits.

For each FRI round $k = 0, 1, \dots, K - 1$:

- (i) Commit the current polynomial: $r_k^{\text{FRI}} = \text{MT}(F_k)$.
- (ii) Absorb: $\mathcal{T}.\text{absorb}(r_k^{\text{FRI}})$.
- (iii) Derive the fold challenge: $\beta_k \leftarrow \mathcal{T}.\text{squeeze}()$.
- (iv) Fold: $F_{k+1} = \text{Fold}(F_k, \beta_k)$. The domain size decreases from 2^{b_k} to $2^{b_{k+1}}$.

After the final round, absorb a hash of the final polynomial: $\mathcal{T}.\text{absorb}(\text{LinearHash}(F_K))$.

Folding operation. **Input:** Polynomial F_k on a domain of size 2^{b_k} , challenge $\beta_k \in \mathbb{F}_{p^3}$. **Output:** Polynomial F_{k+1} on a domain of size $2^{b_{k+1}}$, where $b_{k+1} < b_k$.

Let $f = 2^{b_k - b_{k+1}}$ be the fold factor. For each output index $j \in [2^{b_{k+1}}]$:

- (i) **Gather** f evaluations from F_k :

$$\{F_k[j + i \cdot 2^{b_{k+1}}]\}_{i=0}^{f-1}.$$

- (ii) **Interpolate** to coefficient form $c_0, \dots, c_{f-1}$ (size- f inverse transform).
- (iii) **Coset correction.** For $i = 0, \dots, f - 1$:

$$c_i \leftarrow c_i \cdot (g^{-2^{n_{\text{ext}} - b_k}} \cdot \omega_{b_k}^{-j})^i, \quad (7)$$

where ω_{b_k} is the primitive 2^{b_k} -th root of unity. Note: $b_0 = n_{\text{ext}}$ for the initial round, so $g^{-2^{n_{\text{ext}} - b_0}} = g^{-1}$.

- (iv) **Evaluate** the polynomial $\sum_i c_i \cdot X^i$ at β_k via Horner's method:

$$F_{k+1}[j] = c_0 + c_1 \beta_k + c_2 \beta_k^2 + \dots + c_{f-1} \beta_k^{f-1}.$$

Commitment per round. After each fold, the prover commits F_{k+1} via a Merkle tree. The evaluations are grouped so that each Merkle leaf contains the evaluations that will form one coset group in the next fold step:

- Tree height: $2^{b_{k+2}}$ leaves (anticipating the next fold).
- Each leaf contains $2^{b_{k+1}-b_{k+2}} \cdot 3$ field elements (3 elements per $\mathbb{F}_{p^3}$ value, grouped for the next round's fold factor).

The root $r_{k+1}^{\text{FRI}} = \text{MT}(F_{k+1})$ is absorbed into the transcript.

3.8 Proof of Work

(i) Derive the grinding challenge: $\chi_{\text{grind}} \leftarrow \mathcal{T}.\text{squeeze}()$.

(ii) Find a nonce $\eta \in \mathbb{Z}_{\geq 0}$ such that

$$\text{Poseidon2}(\chi_{\text{grind}} \parallel \eta) \text{ has } b_{\text{pow}} \text{ leading zero bits.}$$

Proof output. The complete proof consists of:

$$\pi = (r_1, r_2, r_Q, \{e_{p,o}\}, \eta, \{r_k^{\text{FRI}}\}_{k=0}^{K-1}, F_K, \{\text{Merkle proofs}\}).$$

4 STARK Protocol: Query Phase

The query phase is the second half of the STARK protocol. The verifier receives the proof π , the verification key vk , the public inputs, and the AIR constraint definitions. It first replays the prover's transcript to rederive all challenges, then performs the checks shown below.

Table 2: PIL2-STARK: Query Phase

Prover	Verifier
<i>Setup: Transcript reconstruction (Section 4.1)</i>	Replay transcript from proof data to rederive all challenges ($\alpha, \gamma, v_c, \xi, v_1, v_2, \{\beta_k\}, \chi_{\text{grind}}$)
<i>Constraint check (Section 4.2)</i>	Evaluate $C(\xi)$ from $\{e_{p,o}\}$ Reconstruct $Q(\xi) = \sum_j \xi^{jN} e_{Q_j,0}$ Check: $Q(\xi) = C(\xi)/Z_H(\xi)$
<i>Proof-of-work check (Section 4.3)</i>	Check: $\text{Poseidon2}(\chi_{\text{grind}} \parallel \eta)$ has b_{pow} leading zeros
<i>Final polynomial degree check (Section 4.4)</i>	$\hat{F}_K = \text{INTT}(F_K)$ Check: $\hat{F}_K[i] = 0$ for $i \geq D$
<i>Query derivation and verification (Section 4.5)</i>	
Both sides hold all commitments, challenges, and η	= Both sides hold all commitments, challenges, and η
$(q_1, \dots, q_{Q_{\text{queries}}})$	$\leftarrow$ Seed $\mathcal{T}'$ from $(\chi_{\text{grind}}, \eta)$ $(q_1, \dots, q_{Q_{\text{queries}}}) \leftarrow \mathcal{T}'.\text{squeeze.indices}(Q_{\text{queries}}, b_0)$

For each query q_i , and each commitment tree (stages $1, 2, Q$, constants, FRI layers $0, \dots, K-1$): compute Merkle opening proof at the appropriate index

→ For each query q_i :
Check: all Merkle proofs verify against committed roots (Section 4.6)
 Let $x_{q_i} = g \cdot \omega_{\text{ext}}^{q_i}$
Check: $F(x_{q_i})$ from batching formula matches FRI layer 0 at q_i (Section 4.7)
 For each FRI round $k = 1, \dots, K$:
 Interpolate coset siblings
 Apply coset correction
 Evaluate at $\hat{\beta}_k$
Check: result matches layer k (Section 4.8)
Accept iff all checks pass

The following subsections detail each check shown in the diagram above.

4.1 Transcript Reconstruction

The verifier replays the prover's Fiat-Shamir transcript to rederive all challenges. The transcript operations must occur in exactly the same order as during proving:

- (i) Seed the transcript (same as prover: multi-AIR or standalone mode).
- (ii) For each stage $s = 2, \dots, S+1$: squeeze the stage- s challenges, then absorb r_s and any intermediate values for stage s .
- (iii) Squeeze the evaluation challenge ξ .
- (iv) Absorb the evaluation hash $\text{LinearHash}(\{e_{p,o}\})$.
- (v) Squeeze the batching challenges v_1, v_2 .
- (vi) For each FRI round $k = 0, \dots, K-1$: absorb r_k^{FRI} and squeeze the fold challenge β_k .
- (vii) Absorb $\text{LinearHash}(F_K)$.
- (viii) Squeeze the grinding challenge χ_{grind} .

4.2 Constraint Check

Verify that the quotient polynomial is consistent with the constraint polynomial.

- (i) Evaluate the combined constraint polynomial at ξ using the claimed evaluations $\{e_{p,o}\}$:

$$C(\xi) = v_c^{J-1} \cdot C_0(\xi) + v_c^{J-2} \cdot C_1(\xi) + \dots + C_{J-1}(\xi),$$

where each $C_j(\xi)$ is computed from the evaluations via the AIR constraint expressions and J is the number of constraints.

- (ii) Compute the vanishing polynomial at ξ :

$$Z_H(\xi) = \xi^N - 1.$$

(iii) Reconstruct the quotient evaluation from the split pieces:

$$Q(\xi) = \sum_{j=0}^{d-1} \xi^{jN} \cdot e_{Q_j,0}. \quad (8)$$

(iv) **Check:**

$$\boxed{Q(\xi) = \frac{C(\xi)}{Z_H(\xi)}}. \quad (9)$$

4.3 Proof-of-Work Check

Check:

Poseidon2($\chi_{\text{grind}} \parallel \eta$) has b_{pow} leading zero bits.

4.4 Final Polynomial Degree Check

- (i) Convert the final polynomial F_K from evaluation form to coefficient form via INTT.
- (ii) Let $D = 2^{b_K - (n_{\text{ext}} - n)}$ be the degree bound.
- (iii) **Check:** all coefficients above degree D are zero:

$$\hat{F}_K[i] = 0 \quad \text{for all } i \geq D.$$

4.5 Query Derivation

- (i) Initialize a fresh transcript $\mathcal{T}'$ and seed it:

$$\mathcal{T}'.\text{absorb}(\chi_{\text{grind}}, \eta).$$

- (ii) Derive Q_{queries} query indices:

$$(q_1, \dots, q_{Q_{\text{queries}}}) \leftarrow \mathcal{T}'.\text{squeeze_indices}(Q_{\text{queries}}, b_0).$$

4.6 Merkle Tree Verification

For each query q and each commitment (stage 1, stage 2, quotient, constants, and any custom commits):

- (i) Hash the leaf values via Poseidon2 linear hashing.
- (ii) Walk the authentication path using the provided siblings.
- (iii) **Check:** the computed root matches the committed root.

4.7 FRI Polynomial Consistency

For each query point $q \in \{q_1, \dots, q_{Q_{\text{queries}}}\}$:

- (i) Let $x_q = g \cdot \omega_{\text{ext}}^q \in H^*$ be the evaluation point.
- (ii) Using the polynomial values extracted from Merkle proofs at index q , compute $F(x_q)$ via the batching formula (6):

$$F(x_q) = \sum_{g \in \mathcal{G}} v_1^{|\mathcal{G}|-1-g} \left(\frac{1}{x_q - \xi \cdot \omega^{o_g}} \sum_{j=0}^{n_g} v_2^{n_g-j} (p_j(x_q) - e_j) \right).$$

- (iii) **Check:** $F(x_q)$ matches the value committed in the first FRI layer at the corresponding index.

4.8 FRI Folding Verification

For each FRI round $k = 1, \dots, K$ and each query q :

- (i) Extract the $f = 2^{b_{k-1}-b_k}$ sibling evaluations from the layer- $(k-1)$ Merkle proof (all members of the coset group containing query q).
- (ii) Interpolate the siblings to coefficient form (size- f interpolation).
- (iii) Apply coset correction and evaluate at the transformed challenge point:

$$\hat{\beta}_k = \frac{\beta_{k-1}}{g^{2^{n_{\text{ext}}-b_{k-1}}} \cdot \omega_{b_{k-1}}^q},$$

where $\omega_{b_{k-1}}$ is the primitive $2^{b_{k-1}}$ -th root of unity and $g^{2^{n_{\text{ext}}-b_{k-1}}}$ is the accumulated coset shift at FRI level $k-1$.

- (iv) Compute the folded value:

$$v = \sum_{i=0}^{f-1} c_i \cdot \hat{\beta}_k^i,$$

where $c_0, \dots, c_{f-1}$ are the interpolated coefficients.

- (v) **Check:** v matches the committed value in layer k (or in the final polynomial F_K for the last round).

Acceptance. The verifier accepts if and only if all checks in Sections 4.2 to 4.4 and 4.6 to 4.8 pass.

5 Multi-AIR Challenge Binding

When a computation is decomposed into multiple AIR instances (e.g. memory, ALU, range checks in a zkVM), each AIR produces an independent STARK proof. To prevent a malicious prover from mixing proofs across different executions, all per-AIR transcripts are seeded with a shared global challenge χ derived from every AIR's stage-1 commitment.

5.1 Per-AIR Contributions

Suppose there are A AIR instances. Each instance $a \in [A]$ independently commits its stage-1 polynomials to obtain a Merkle root $r_1^{(a)}$. It then computes a *contribution* $\kappa^{(a)}$ as follows.

- (i) Assemble input data:

$$\mathbf{d}^{(a)} = (\text{vk}^{(a)}, r_1^{(a)}, \text{airValues}^{(a)}),$$

where $\text{vk}^{(a)}$ is the per-AIR verification key (4 base-field elements) and $\text{airValues}^{(a)}$ are any stage-1 air values.

- (ii) Compute a hash state:

$$h^{(a)} = \text{Poseidon2}(\mathbf{d}^{(a)}) \in \mathbb{F}_p^{16}.$$

- (iii) **Lattice expansion** (when no elliptic-curve mode is used). Chain-hash to produce a vector of L elements (typically $L = 368$):

$$\begin{aligned} \kappa^{(a)}[0:16] &= h^{(a)}, \\ \kappa^{(a)}[16k:16(k+1)] &= \text{Poseidon2}(\kappa^{(a)}[16(k-1):16k]) \quad \text{for } k = 1, \dots, L/16 - 1. \end{aligned}$$

5.2 Challenge Aggregation

The partial contributions are aggregated and a shared challenge is derived.

Lattice mode. Sum all contribution vectors elementwise:

$$\kappa = \sum_{a=0}^{A-1} \kappa^{(a)} \in \mathbb{F}_p^L.$$

Elliptic-curve mode. Each contribution is mapped to a point on an elliptic curve $E/\mathbb{F}_{p^5}$ via a hash-to-curve map, then all points are summed:

$$P = \sum_{a=0}^{A-1} \text{HashToCurve}(\kappa^{(a)}[0:5], \kappa^{(a)}[5:10]) \in E(\mathbb{F}_{p^5}).$$

The aggregated value is $(P.x \parallel P.y) \in \mathbb{F}_p^{10}$.

Global challenge derivation. From the aggregated value κ (or the curve-mode representation), derive the global challenge:

- (i) Initialize a fresh transcript $\mathcal{T}_G$.
- (ii) $\mathcal{T}_G.\text{absorb}(\text{publics})$.
- (iii) $\mathcal{T}_G.\text{absorb}(\text{proofValues}_{\text{stage 1}})$ (if any).
- (iv) $\mathcal{T}_G.\text{absorb}(\kappa)$.
- (v) Squeeze: $\chi \leftarrow \mathcal{T}_G.\text{squeeze}()$, yielding $\chi \in \mathbb{F}_{p^3}$.

Every AIR instance then uses χ as its transcript seed (as described in Section 3.2), binding all per-AIR proofs to the same shared randomness.

5.3 Global Constraints

After all per-AIR proofs are generated, the system verifies cross-AIR consistency via *global constraints*. These enforce that airgroup values accumulated during individual AIR executions (such as gsum and gprod boundary values) satisfy the required aggregate relationships:

- **Sum type (logup):** the sum of all gsum boundary values across AIRs sharing a bus equals zero.
- **Product type (permutation):** the product of all gprod boundary values across AIRs sharing a bus equals one.

The concrete global constraints for a specific machine (e.g. bus balance equations, continuation anchoring) are defined by the machine specification. For the verification circuit that enforces these constraints, see the *Recursion Pipeline* (VadcopFinal).

Part II

Zisk Machine

6 Architecture Overview

Zisk is a zero-knowledge virtual machine that proves correct execution of RISC-V programs. It decomposes computation into a single *airgroup* named “Zisk” containing 21 specialized AIRs (Algebraic Intermediate Representations), each responsible for a different aspect of execution: instruction dispatch, memory, arithmetic, bitwise operations, cryptographic precompiles, and lookup tables.

The AIRs communicate via *buses*—shared lookup and permutation arguments that enforce consistency between components. The central interconnect is the *operation bus*: the Main CPU AIR dispatches operations to coprocessors (Binary, Arith, etc.) via lookup assumes, and each coprocessor proves correctness via lookup proves on the same bus.

6.1 Airgroup Structure

All 21 AIRs belong to a single airgroup (“Zisk”). The aggregation type is **sum** (logup) at stage 2, meaning the global constraint checks that the sum of all gsum boundary values across all AIRs equals zero.

Configuration parameters:

- Lattice size: $L = 368$ (no elliptic-curve mode).
- Transcript arity: $a = 4$ (Poseidon2 sponge width = 16).
- Number of public inputs: 68 (4 for `rom_root` + 64 for `inputs`).
- Stage-2 challenges: 2 (α, γ).
- Proof values: 2 at stage 1 (`enable_input_data`, `enable_rom_data`).

6.2 AIR Inventory

ID	Name	Trace size	Role	Compressor?
0	Main	2^{22}	CPU instruction dispatch	No
1	Rom	2^{22}	Program ROM lookup table	No
2	Mem	2^{22}	Main memory (sorted by addr+step)	No
3	RomData	2^{21}	Immutable ROM data region	No
4	InputData	2^{21}	Free input data region	No
5	MemAlign	2^{21}	Unaligned memory access logic	No
6	MemAlignByte	2^{22}	Byte-level memory alignment	No
7	MemAlignReadByte	2^{22}	Read-side byte alignment	No
8	MemAlignWriteByte	2^{22}	Write-side byte alignment	No
9	Arith	2^{21}	64-bit multiply/divide	No
10	Binary	2^{22}	Bitwise AND, OR, XOR, comparisons	No
11	BinaryAdd	2^{22}	Dedicated 64-bit addition	No
12	BinaryExtension	2^{22}	Shifts, sign-extension	No
13	Add256	2^{20}	256-bit addition	No
14	ArithEq	2^{20}	256-bit field arithmetic (secp256k1, BN254)	Yes
15	ArithEq384	2^{20}	384-bit field arithmetic (BLS12-381)	Yes
16	Keccakf	2^{17}	Keccak-f[1600] permutation	Yes
17	Sha256f	2^{18}	SHA-256 compression function	Yes
18	SpecifiedRanges	2^{20}	Range check lookup table	No
19	VirtualTable0	2^{21}	Packed lookup tables (7 tables)	No
20	VirtualTable1	2^{21}	Packed lookup tables (3 tables)	No

AIRs marked “Compressor = Yes” have STARK verifier circuits exceeding 2^{17} rows, requiring the optional Compressor stage in the recursion pipeline (see the *Recursion Pipeline*).

6.3 STARK Parameters per AIR

Each AIR defines a STARK instance with concrete parameters. All AIRs share: blowup factor $\beta = 2$, Merkle arity $a = 4$, quotient degree $d = 2$ (except Rom with $d = 1$), and cubic extension $\mathbb{F}_{p^3}$ ($q_{\text{dim}} = 3$).

Name	n	Witness	Intmd	Const	Eval pts	FRI rounds
Main	22	38	8	3	61	7
Rom	22	1	2	1	18	7
Mem	22	13	3	2	29	7
RomData	21	6	3	2	19	7
InputData	21	9	6	2	27	7
MemAlign	21	29	6	2	59	7
MemAlignByte	22	16	4	1	25	7
MemAlignReadByte	22	10	3	1	18	7
MemAlignWriteByte	22	14	4	1	23	7
Arith	21	44	15	1	64	7
Binary	22	39	5	1	49	7
BinaryAdd	22	10	3	1	18	7
BinaryExtension	22	29	6	1	40	7
Add256	20	47	17	1	69	6
ArithEq	20	39	14	2	434	6
ArithEq384	20	33	14	2	536	6
Keccakf	17	2137	293	2	4065	5
Sha256f	18	102	7	2	1265	6
SpecifiedRanges	20	33	17	34	88	6
VirtualTable0	21	8	5	52	69	7
VirtualTable1	21	8	5	73	90	7

Columns: n = trace size exponent ($N = 2^n$), Witness = stage-1 committed columns, Intmd = stage-2 intermediate columns, Const = constant (setup) columns, Eval pts = evaluation map entries (polynomial openings in the FRI batching), FRI rounds = number of folding rounds. The number of FRI queries ranges from 217 (Keccakf) to 232 (ArithEq384), targeting approximately 100 bits of security.

Notable outliers. Keccakf has by far the most columns (2137 witness + 293 intermediate) because each Keccak-f round requires extensive bit-level decomposition. ArithEq384 has the most evaluation points (536) due to the large number of constraint polynomials in its 384-bit modular arithmetic verification. The lookup table AIRs (SpecifiedRanges, VirtualTable0, VirtualTable1) have many constant columns (up to 73) because the precomputed table values are committed as constant polynomials during setup.

7 Bus Architecture

7.1 Bus Types

Zisk uses three types of inter-AIR communication:

- **Lookup (logup sum-check).** One side *proves* rows (provides entries with multiplicities), the other side *assumes* rows (consumes entries with selectors). The logup argument ensures every assumed tuple was proved.
- **Permutation (multiset equality).** Two sides prove/assume with 1:1 matching. The grand product argument ensures the multisets are identical.

- **Direct update.** Degree-0 constraints for segment chaining and global anchors. Used for continuation across execution segments.

7.2 Bus Inventory

ID	Name	Type	Description
5000	Operation Bus	Lookup	CPU $\leftrightarrow$ coprocessors
7890	ROM Bus	Lookup	Main $\leftrightarrow$ Rom
10	Memory Bus	Permutation	Memory subsystem interconnect
88	Dual Byte Table	Lookup	Byte-pair decomposition
125	Binary Table	Lookup	Binary operation lookup
124	Binary Extension Table	Lookup	Shift/extension lookup
126	Keccakf Table	Lookup	Keccak round constant lookup
133	MemAlign ROM	Lookup	Memory alignment microcode
330	Arith Range Table	Lookup	Arithmetic range check
331	Arith Table	Lookup	Arithmetic operation lookup
5002	ArithEq Lt Table	Lookup	Field comparison lookup
5010	Arith Frops Table	Lookup	Arith frequent operations
5011	Binary Frops Table	Lookup	Binary frequent operations
5012	BinaryExt Frops Table	Lookup	BinaryExt frequent operations

7.3 Bus Interconnection Summary

The central bus topology is:

1. **Main CPU** assumes operations on the *Operation Bus* (5000). Every coprocessor (Binary, BinaryAdd, BinaryExtension, Arith, Add256, ArithEq, ArithEq384, Keccakf, Sha256f) proves on this bus.
2. **Main CPU** assumes instruction fetches on the *ROM Bus* (7890). The **Rom** AIR proves instruction tuples. **RomData** also assumes on the ROM Bus for data reads.
3. **Memory operations** use the *Memory Bus* (10) with permutation arguments. Main, Mem, RomData, InputData, MemAlign, and the MemAlign byte variants all participate.
4. **Lookup tables** (SpecifiedRanges, VirtualTable0, VirtualTable1) prove on their respective table buses; computation AIRs assume range checks and operation tables.

8 CPU: Main AIR

The Main AIR (2^{22} rows) is the central instruction dispatch unit.

8.1 Instruction Execution

Each row of the Main trace represents one clock cycle. The Main AIR decodes the current instruction and dispatches the operation to the appropriate coprocessor via the Operation Bus. It does not perform computation directly—all arithmetic, bitwise, and cryptographic operations are delegated.

8.2 Bus Interactions

- **Operation Bus (5000), assumes.** Dispatches operations with the tuple (opcode, step, 0, addr, 0, ...). The coprocessor that handles this opcode proves the matching tuple.
- **ROM Bus (7890), assumes.** Fetches instruction words from the Rom AIR. The tuple includes program counter, immediate values, and operand encodings.
- **Memory Bus (10), permutation.** Issues memory loads and stores. The Mem AIR proves the corresponding memory operations.
- **Operation Bus (5000), direct update.** Provides continuation anchoring: the initial state (boot address) and final state (end address) are published as direct global updates, checked by the global constraints.

9 Memory Subsystem

9.1 Mem AIR

The Mem AIR (2^{22} rows) implements the main read/write memory. Memory accesses are sorted by (*address, step*) pairs, enabling transition constraints that verify temporal consistency:

- A write at step t stores a value.
- A read at step $t' > t$ at the same address returns the most recently written value.

Mem supports both 1-byte and 8-byte (aligned) accesses. The memory bus uses a permutation argument (bus ID 10) to match loads/stores from the Main AIR and MemAlign subsystem.

9.2 RomData AIR

The RomData AIR (2^{21} rows) holds immutable program data. It follows a *first-write-then-read-only* pattern:

- Data is written once during initialization.
- All subsequent accesses are reads that return the initialized value.
- Proved via both the Memory Bus (10) and ROM Bus (7890).

Gated by the `enable_rom_data` proof value.

9.3 InputData AIR

The InputData AIR (2^{21} rows) provides free input to the computation. It follows a *first-read-determines-value* pattern:

- The first read at an address determines the value (prover's free input).
- Subsequent reads return the same value.
- Connected via the Memory Bus (10).

Gated by the `enable_input_data` proof value.

9.4 MemAlign Subsystem

The MemAlign AIR (2^{21} rows) and its three byte-level variants (MemAlignByte, MemAlignReadByte, MemAlignWriteByte, each 2^{22} rows) form a microprocessor for unaligned memory accesses.

When the Main AIR encounters a memory access that is not naturally aligned, it delegates to the MemAlign subsystem, which:

1. Decomposes the unaligned access into aligned sub-accesses.
2. Performs byte-level read and write operations.
3. Reassembles the result for the Main AIR.

The MemAlign ROM (bus ID 133) provides microcode for the decomposition.

10 Computation Coprocessors

Each coprocessor proves operations on the Operation Bus (5000). The Main AIR assumes tuples of the form $(\text{opcode}, a_1, a_2, \dots, a_k)$, and the coprocessor proves matching tuples with the computed result.

10.1 Binary

The Binary AIR (2^{22} rows) handles bitwise operations: AND, OR, XOR, and unsigned/signed comparisons (LTU, LT). Operations are decomposed into byte-level lookups via the Binary Table (bus ID 125).

10.2 BinaryAdd

The BinaryAdd AIR (2^{22} rows) provides dedicated 64-bit addition. Separated from the Binary AIR for constraint degree optimization.

10.3 BinaryExtension

The BinaryExtension AIR (2^{22} rows) handles shifts (logical left/right, arithmetic right) and sign-extension operations. Uses the Binary Extension Table (bus ID 124) for byte-level decomposition.

10.4 Arith

The Arith AIR (2^{21} rows) handles 64-bit multiplication and division. Operands are decomposed into 16-bit chunks, and the multiplication is verified via a schoolbook decomposition with carry propagation. Uses the Arith Table (bus ID 331) and Arith Range Table (bus ID 330) for chunk-level lookups.

11 Precompiles

Precompile AIRs provide accelerated circuits for cryptographic operations. All four precompile AIRs require the Compressor stage in the recursion pipeline due to their large verifier circuits.

11.1 Add256

The Add256 AIR (2^{20} rows) provides 256-bit unsigned addition. Used as a building block for multi-precision arithmetic.

11.2 ArithEq

The ArithEq AIR (2^{20} rows) provides 256-bit modular arithmetic for elliptic curve operations over secp256k1 and BN254. Supports:

- 256-bit modular addition, subtraction, and multiplication.
- secp256k1 point addition and doubling.
- BN254 curve operations (addition, doubling, complex field arithmetic).

Uses the ArithEq Lt Table (bus ID 5002) for field comparison lookups.

11.3 ArithEq384

The ArithEq384 AIR (2^{20} rows) extends ArithEq to 384-bit fields, supporting BLS12-381 elliptic curve operations.

11.4 Keccakf

The Keccakf AIR (2^{17} rows) implements the Keccak-f[1600] permutation, the core of the Keccak/SHA-3 hash function. Each invocation processes a 1600-bit state through 24 rounds. Memory operations for reading/writing the state use the Memory Bus (10). Uses the Keccakf Table (bus ID 126) for round constant lookups.

11.5 Sha256f

The Sha256f AIR (2^{18} rows) implements the SHA-256 compression function. Each invocation processes a 512-bit message block with a 256-bit state. Memory operations use the Memory Bus (10).

12 Lookup Tables

12.1 SpecifiedRanges

The SpecifiedRanges AIR (2^{20} rows) provides range check lookups. It proves that values fall within specified ranges, used by various AIRs for constraint verification.

12.2 VirtualTable0

The VirtualTable0 AIR (2^{21} rows) packs 7 lookup tables into a single AIR for efficiency:

Bus ID	Table	Consumer
331	Arith Table	Arith
330	Arith Range Table	Arith
5002	ArithEq Lt Table	ArithEq, ArithEq384
124	Binary Extension Table	BinaryExtension
125	Binary Table	Binary
133	MemAlign ROM	MemAlign
126	Keccakf Table	Keccakf

12.3 VirtualTable1

The VirtualTable1 AIR (2^{21} rows) packs 3 frequent-operation tables:

Bus ID	Table	Consumer
5010	Arith Frops Table	Arith
5011	Binary Frops Table	Binary
5012	BinaryExt Frops Table	BinaryExtension

These “frequent operations” tables precompute common sub-expressions used by the computation coprocessors to reduce constraint degree.

13 Global Constraints

After all per-AIR proofs are generated, the system must verify *cross-AIR consistency*: that buses balance, that the execution begins and ends at the correct state, and that public inputs

match. This verification is performed by the VadcopFinal circuit (see the *Recursion Pipeline*), which encodes the global constraints as a single compiled constraint expression.

13.1 Bus Balance Equations

Each bus connects two or more AIRs via a logup or permutation argument. Each participating AIR accumulates a running sum (gsum) or running product (gprod) during its own STARK proof. The *boundary value* $\text{gsum}^{(a)}[N^{(a)} - 1]$ (the final row of the running sum for AIR a) is exported as an *airgroup value* and carried through the Recursive2 aggregation tree.

For each bus, the global constraint checks that the boundary values from all participating AIRs cancel:

- **Lookup buses** (Operation Bus, ROM Bus, all table buses):

$$\sum_{a \in \text{bus}(b)} \text{gsum}^{(a)}[N^{(a)} - 1] = 0.$$

Providers contribute positive multiplicities and consumers contribute negative selectors; if every consumed tuple was provided, the sum is zero.

- **Permutation buses** (Memory Bus):

$$\prod_{a \in \text{bus}(b)} \text{gprod}^{(a)}[N^{(a)} - 1] = 1.$$

The two sides form identical multisets; the grand product ratio equals one.

Concretely, Zisk has 14 buses (Section 7.2), producing 14 balance equations that the global constraint must enforce. The single compiled global constraint expression (53,890 bytecode operations) evaluates all 14 balance equations simultaneously, verifying the entire bus fabric in one check.

13.2 Continuation Anchoring

Direct-update constraints anchor the execution to known boundary values:

- **Main continuation.** The initial program counter equals the boot address. The final program counter equals the designated end address. Published via direct global updates on the Operation Bus.
- **Memory continuation.** Memory base addresses and final states are anchored.
- **Public output.** The 64 public input/output values (`inputs[0..63]`) are published via direct global updates, binding the execution trace to the claimed public inputs.

13.3 Airgroup Value Aggregation

The single airgroup uses sum-type aggregation at stage 2. During recursive aggregation (see *Recursion Pipeline*, Recursive2), per-AIR gsum boundary values are added together. VadcopFinal checks that the final aggregated sum equals zero for each bus, confirming cross-AIR consistency.

The aggregation flow is:

1. Each AIR computes $\text{gsum}^{(a)}[N - 1] \in \mathbb{F}_{p^3}$ as a boundary value of its running logup sum.
2. Recursive2 adds boundary values from child proofs:

$$\text{gsum}_{\text{agg}} = \sum_{a \in \text{children}} \text{gsum}^{(a)}.$$

3. VadcopFinal receives the fully aggregated sum and verifies:

$$\text{gsum}_{\text{agg}} = 0 \quad (\text{for each bus}).$$

Part III

Recursion Pipeline

14 Pipeline Overview

A multi-AIR computation produces A independent STARK proofs, one per AIR instance. The aggregation pipeline transforms these into a single proof through up to seven stages:

Stage	Name	Input	Output
1	Basic	AIR witness	Per-AIR STARK proof
2	Compressor	Basic STARK proof	Compressed STARK proof
3	Recursive1	Basic/Compressor	Normalized SNARK proof
4	Recursive2	≤ 3 Recursive1/2	Aggregated SNARK proof
5	VadcopFinal	One R2 per airgroup	Cross-AIR verified STARK proof
<i>The following stages are not currently deployed (see note below):</i>			
6	RecursiveF	VadcopFinal	BN254-native STARK proof
7	fflonk	RecursiveF	Ethereum-verifiable SNARK

Stages 1–5 operate entirely over the Goldilocks field $\mathbb{F}_p$ with Poseidon2 hashing. Stage 6 would transition to BN254 with Poseidon hashing over the BN254 scalar field. Stage 7 would produce a pairing-based SNARK verifiable by an on-chain Solidity contract.

Current deployment. In the current Zisk ethproofs integration, the pipeline terminates at Stage 5 (VadcopFinal). The coordinator sets `final.snark=false`, which skips Stages 6 and 7 entirely. The VadcopFinal STARK proof—a Goldilocks-native proof with no elliptic curve cryptography—is submitted directly to the ethproofs aggregation service. Stages 6 and 7 (RecursiveF and fflonk) are implemented but reserved for a future configuration where on-chain STARK verification is replaced by a constant-size pairing-based SNARK.

14.1 Proof Type Table

Type	System	Description
Basic	STARK	Per-AIR STARK proof (see the <i>STARK Protocol</i>)
Compressor	STARK	Optional compression for large Basic proofs
Recursive1	Circom SNARK	Wraps one Basic/Compressor STARK in a Groth16-style circuit
Recursive2	Circom SNARK	Aggregates ≤ 3 proofs of the same airgroup into 1 (tree reduction)
VadcopFinal	STARK	Combines one Recursive2 per airgroup; verifies global challenge and global constraints
RecursiveF	STARK	Re-proves VadcopFinal over BN254 for pairing-friendly verification
fflonk	SNARK	Ethereum-verifiable proof (fflonk variant of PLONK)

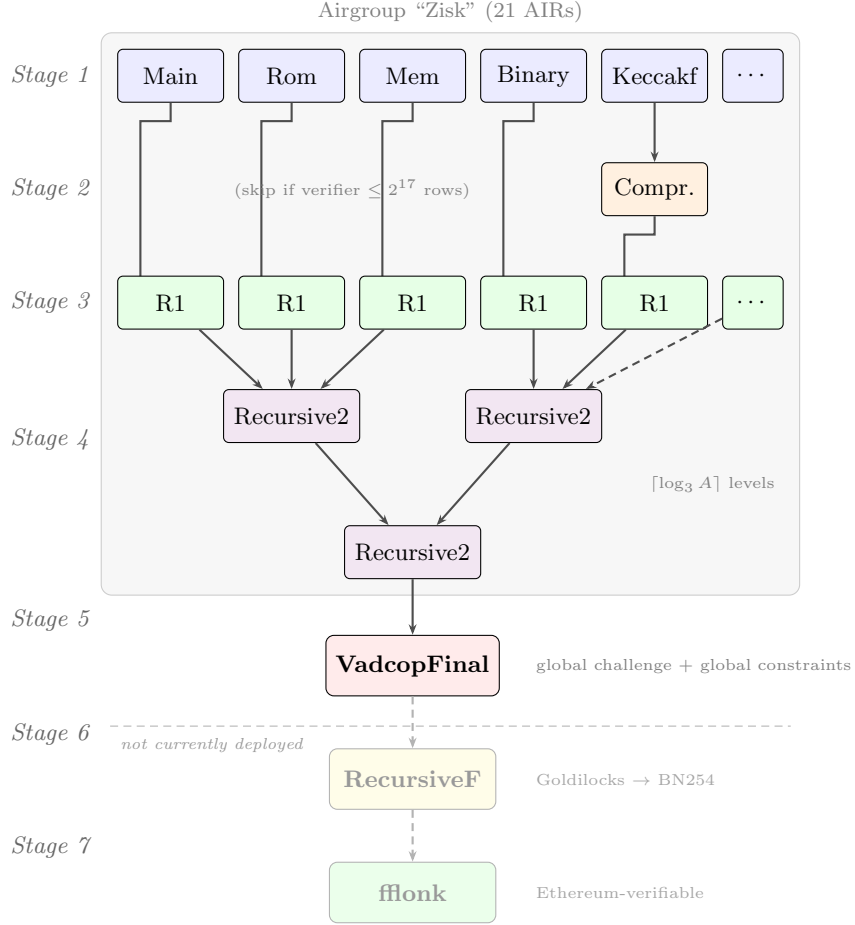


Figure 1: VADCOP recursive aggregation pipeline. Each per-AIR Basic proof is optionally compressed, normalized by Recursive1, then aggregated via a Recursive2 tree within each air-group. VadcopFinal verifies the global challenge and cross-AIR constraints. Stages below the dashed line (RecursiveF, fflonk) are not currently deployed.

14.2 Field Transitions

- **Stages 1–5** (Basic through VadcopFinal): Goldilocks field $\mathbb{F}_p = \mathbb{F}_{2^{64}-2^{32}+1}$, Poseidon2 hashing, cubic extension $\mathbb{F}_{p^3}$. In the current deployment, the entire pipeline uses this single field.
- **Stage 6** (RecursiveF, not deployed): BN254 scalar field $\mathbb{F}_r$, where r is the order of the BN254 curve group. Poseidon hashing over $\mathbb{F}_r$. Goldilocks arithmetic is emulated within BN254 circuits.
- **Stage 7** (fflonk, not deployed): BN254 pairing-based SNARK. The output would be a constant-size proof verifiable by a Solidity smart contract.

15 Basic STARK Proof (Stage 1)

Each AIR instance produces an independent STARK proof using the protocol described in the *STARK Protocol*.

- The prover commits stage-1 polynomials and computes a contribution $\kappa^{(a)}$ for challenge aggregation (see the *STARK Protocol*, Multi-AIR Challenge Binding).
- After the global challenge χ is derived from all contributions (see the *STARK Protocol*,

Challenge Aggregation), each AIR seeds its transcript with χ and runs the full commitment and query phases.

- (iii) The output is a binary proof $\pi_{\text{Basic}}^{(a)}$ containing Merkle roots, polynomial evaluations, FRI layers, proof-of-work nonce, and Merkle opening proofs.

16 Compressor (Stage 2, Optional)

The Compressor is an optional stage applied when the STARK verifier circuit for a particular AIR exceeds 2^{17} rows.

- (i) **Circuit.** A Circom circuit over the Goldilocks field that verifies one Basic STARK proof. The circuit’s execution trace forms a new AIR.
- (ii) **When needed.** If the number of verifier circuit rows is $\leq 2^{17}$, the Compressor is skipped and the Basic proof proceeds directly to Recursive1.
- (iii) **Output.** A new STARK proof $\pi_{\text{Comp}}^{(a)}$ with a smaller verification circuit than the original Basic proof.
- (iv) **stage1Hash.** The Compressor computes a commitment hash:

$$\text{stage1Hash}^{(a)} = \text{Poseidon2}(\text{vk}^{(a)}, r_1^{(a)}, \text{airValues}^{(a)}),$$

which chains through subsequent recursive layers.

17 Recursive1: Per-AIR Normalization (Stage 3)

Recursive1 wraps each Basic or Compressor STARK proof in a SNARK circuit, producing a normalized proof format suitable for tree aggregation.

- (i) **Circuit.** A Circom SNARK circuit that verifies one STARK proof. The verification key is selected based on the input proof’s circuit type:
 - Circuit type = “basic”: use the Basic STARK verification key.
 - Circuit type = “compressor”: use the Compressor verification key.
- (ii) **Input.** One STARK proof $\pi^{(a)}$ (either Basic or Compressor).
- (iii) **Output.** A normalized proof $\pi_{\text{R1}}^{(a)}$ with a standardized output format containing:
 - Public inputs and proof values.
 - The global challenge χ .
 - The root of the aggregated challenge computation rootCAgg .
 - The stage1Hash for this AIR instance.

18 Recursive2: Tree Aggregation (Stage 4)

Recursive2 aggregates proofs within the same airgroup using a 3-to-1 tree reduction.

18.1 Aggregation Structure

- (i) **Fan-in.** Each Recursive2 circuit takes up to $N_{\text{agg}} = 3$ input proofs. These may be Recursive1 proofs or other Recursive2 proofs from the same airgroup.
- (ii) **Null-proof padding.** If the number of proofs is not divisible by 3, null proofs are used as padding. The circuit distinguishes null proofs via a circuit type flag (circuit type = 0 indicates a null proof that is not verified).
- (iii) **Verification key selection.** The circuit selects the appropriate verification key based on the circuit type of each input:
 - Type 0: null proof (skip verification).
 - Type 1: aggregated proof (Recursive2 verification key).
 - Type ≥ 2 : basic proof (Recursive1 verification key, with type encoding the specific AIR).

18.2 Airgroup Value Aggregation

Each input proof carries airgroup values (accumulated gsum and gprod boundary values). Recursive2 aggregates these:

- **Sum type** (gsum): add the values from all non-null inputs.
- **Product type** (gprod): multiply the values from all non-null inputs.

18.3 stage1Hash Chaining

The stage1Hash values from input proofs are chained using Poseidon2:

$$\text{stage1Hash}_{\text{out}} = \text{Poseidon2}(\text{stage1Hash}_A \parallel \text{stage1Hash}_B).$$

For three inputs A, B, C : first combine A and B , then combine the result with C . Null proofs contribute a zero hash.

18.4 Tree Depth

For an airgroup with N_{AIR} instances, the tree has depth

$$D = \lceil \log_3(N_{\text{AIR}}) \rceil.$$

The output is a single proof $\pi_{\text{R2}}^{(g)}$ per airgroup g .

19 VadcopFinal (Stage 5)

The VadcopFinal circuit is itself an AIR whose constraints enforce cross-airgroup consistency. It takes one Recursive2 proof per airgroup and produces a single STARK proof.

19.1 STARK Verification

For each airgroup g , the VadcopFinal circuit verifies the Recursive2 proof $\pi_{\text{R2}}^{(g)}$ using the appropriate verification key (selected between Basic and aggregation keys depending on the circuit type flag).

19.2 Global Challenge Recomputation

The circuit recomputes the global challenge from the public inputs:

$$\chi' = \mathcal{T}_G(\text{publics}, \text{proofValues}, \text{stage1Hash}^{(0)}, \dots, \text{stage1Hash}^{(G-1)}),$$

where each $\text{stage1Hash}^{(g)}$ is the chained Poseidon2 hash accumulated through the Recursive2 tree for airgroup g .

Check: $\chi' = \chi$ (the global challenge used by all per-AIR proofs).

19.3 Global Constraint Verification

The circuit enforces all cross-airgroup constraints. For a concrete machine, these are defined by the machine specification (e.g. bus balance equations, continuation anchoring; see the *Zisk Machine* specification).

In general, global constraints take two forms:

- **Sum type (logup):** the sum of all gsum boundary values across AIRs sharing a bus equals zero.

$$\sum_{a \in \text{bus}(b)} \text{gsum}^{(a)}[N-1] = 0 \quad \text{for each bus } b.$$

- **Product type (permutation):** the product of all gprod boundary values across AIRs sharing a bus equals one.

$$\prod_{a \in \text{bus}(b)} \text{gprod}^{(a)}[N-1] = 1 \quad \text{for each bus } b.$$

19.4 Output

The VadcopFinal proof π_{VF} is itself a STARK proof over the Goldilocks field, verified by the standard STARK verifier (see the *STARK Protocol*, Query Phase). Its transcript is seeded directly with $(\text{vk}, \text{H}(\text{pub}), r_1)$ rather than with a global challenge, since it *is* the outermost layer.

In the current deployment, this is the **final proof** submitted to the ethproofs service. The proof is serialized as a flat array of Goldilocks field elements (**Vec<u64>**) and transmitted as base64-encoded binary.

20 RecursiveF: Field Transition (Stage 6, Not Deployed)

This stage is implemented but not currently deployed. The pipeline terminates at VadcopFinal (Stage 5) in the current configuration.

RecursiveF re-proves the VadcopFinal STARK over the BN254 scalar field, enabling verification by a pairing-based SNARK in the final stage.

- (i) **Purpose.** The Goldilocks-based STARK proof from VadcopFinal cannot be directly verified by a BN254 pairing check. RecursiveF “translates” the proof to BN254-native arithmetic.
- (ii) **BN254-native Merkle hashing.** The Merkle trees in the RecursiveF proof use Poseidon over the BN254 scalar field $\mathbb{F}_r$ instead of Poseidon2 over Goldilocks.
- (iii) **Goldilocks emulation.** Goldilocks field arithmetic ($p = 2^{64} - 2^{32} + 1$) is emulated within BN254 circuits. This is the computational bottleneck of the recursion pipeline.
- (iv) **Output.** A STARK proof π_{RF} over BN254, ready for the fflonk stage.

21 fflonk: Ethereum-Verifiable Proof (Stage 7, Not Deployed)

This stage is implemented but not currently deployed. The pipeline terminates at VadcopFinal (Stage 5) in the current configuration.

The final stage would produce a constant-size proof verifiable on Ethereum.

- (i) **System.** A BN254 SNARK using the fflonk variant of PLONK, which requires only one pairing check for verification.
- (ii) **publicsHash.** The public inputs are hashed via SHA-256 to produce a single publicsHash value that the on-chain verifier checks.
- (iii) **On-chain verifier.** A Solidity smart contract that:
 - (a) Recomputes publicsHash from the provided public inputs.
 - (b) Verifies the fflonk proof using the BN254 precompiles (ecAdd, ecMul, ecPairing).
- (iv) **Output.** A constant-size proof π_{fflonk} (typically ~ 768 bytes) plus the public inputs.

22 Distributed Proving

The VADCOP architecture enables distributed proving: workers independently compute per-AIR proofs and contribute to challenge aggregation without sharing witness data. The three-phase workflow is:

- Phase 1: Contributions.** Each worker computes stage-1 commitments and lattice contributions for its assigned AIR instances. Contributions are sent to the coordinator.
- Phase 2: Prove.** The coordinator aggregates all contributions into the global challenge χ and distributes it to all workers. Workers generate per-AIR STARK proofs (Basic stage) and perform internal Recursive1/Recursive2 aggregation for their assigned airgroups.
- Phase 3: Aggregate.** A designated aggregator receives all per-airgroup Recursive2 proofs, performs final Recursive2 tree reduction (if needed), and generates the VadcopFinal proof. In the current deployment this is the terminal stage; when `final_snark` is enabled, RecursiveF and fflonk would follow sequentially.

MPI worker aggregation. In the MPI deployment model, each worker process handles a subset of AIR instances. The coordinator process (rank 0) performs the aggregation of contributions and final proof assembly.

Coordinator-mediated multi-machine proving. For large computations spanning multiple machines, the coordinator mediates challenge distribution and proof collection over the network. Each machine runs an independent set of workers, and only the aggregated contributions and final proofs traverse machine boundaries.

Part IV

Appendices

A Constraint Polynomial Structure

Each AIR (Algebraic Intermediate Representation) defines a set of constraint polynomials $C_0, C_1, \dots, C_{J-1}$. Each C_j is a polynomial expression over:

- Committed columns $f_i(X)$, $f_i(X \cdot \omega)$, $f_i(X \cdot \omega^{-1})$ (current, next, and previous row evaluations);
- Constant polynomials $c_i(X)$;
- Challenges $\alpha, \gamma, \dots \in \mathbb{F}_{p^3}$.

Constraint types.

- *Transition constraints*: relate row i to row $i + 1$, e.g. $f(X \cdot \omega) - f(X) - 1 = 0$.
- *Boundary constraints*: fix specific rows, typically row 0 or row $N - 1$, enforced via Lagrange polynomials $L_0(X)$, $L_{N-1}(X)$.
- *Lookup / permutation constraints*: grand-sum or grand-product accumulation polynomials using compressed expressions $(\text{col}_2 \cdot \alpha + \text{col}_1) \cdot \alpha + \text{busid} + \gamma$.

Combination. The individual constraints are combined into a single polynomial using a random challenge $v_c \in \mathbb{F}_{p^3}$ via Horner's method:

$$C(X) = (\dots((C_0 \cdot v_c + C_1) \cdot v_c + C_2) \cdot v_c + \dots) + C_{J-1} = \sum_{j=0}^{J-1} v_c^{J-1-j} \cdot C_j(X). \quad (10)$$

If the AIR is satisfied, then $C(x) = 0$ for all $x \in H$, meaning $Z_H(X) = X^N - 1$ divides $C(X)$ and the quotient $Q(X) = C(X)/Z_H(X)$ is a polynomial.

B FRI Polynomial Batching Formula

The FRI polynomial F batches all polynomial openings into a single polynomial using challenges $v_1, v_2 \in \mathbb{F}_{p^3}$.

Setup. The evaluation map specifies triples (p_i, o_i, e_i) where p_i is a committed polynomial, $o_i \in \mathcal{O}$ is an opening offset, and $e_i = p_i(\xi \cdot \omega^{o_i})$ is the claimed evaluation.

Group these triples by opening offset index. Let $\mathcal{G} = \{g_0, g_1, \dots, g_{|\mathcal{G}|-1}\}$ be the ordered set of distinct opening indices, and for each $g \in \mathcal{G}$ let $(p_{g,0}, e_{g,0}), \dots, (p_{g,n_g}, e_{g,n_g})$ be the entries in that group (in evaluation-map order).

Intra-group accumulation (Horner with v_2). For each group g and evaluation point $x \in H^*$:

$$\hat{G}_g(x) = v_2^{n_g} (p_{g,0}(x) - e_{g,0}) + v_2^{n_g-1} (p_{g,1}(x) - e_{g,1}) + \dots + (p_{g,n_g}(x) - e_{g,n_g}). \quad (11)$$

This is computed via Horner's method: start with accumulator $A = 0$; for each entry $(p_{g,j}, e_{g,j})$ in order, set $A \leftarrow A \cdot v_2 + (p_{g,j}(x) - e_{g,j})$.

Then divide by the DEEP quotient denominator:

$$G_g(x) = \frac{\hat{G}_g(x)}{x - \xi \cdot \omega^{o_g}}.$$

Inter-group accumulation (Horner with v_1). Combine groups:

$$F(x) = v_1^{|\mathcal{G}|-1} \cdot G_{g_0}(x) + v_1^{|\mathcal{G}|-2} \cdot G_{g_1}(x) + \dots + G_{g_{|\mathcal{G}|-1}}(x). \quad (12)$$

Again via Horner: start with $A = 0$; for each group g_i in order, set $A \leftarrow A \cdot v_1 + G_{g_i}(x)$.

Result. F is evaluated on all of H^* to produce the FRI input polynomial of degree $< N_{\text{ext}}$.